

Raft on SQLite: Evaluating the Impact of Consistency and Replication on Performance

Arjun Anil
University of Wisconsin-Madison
aanil3@wisc.edu

Chenglong Yu
University of Wisconsin-Madison
cyu297@wisc.edu

Daniel Ott
University of Wisconsin-Madison
dcott@wisc.edu

Abstract

This project evaluates the performance and correctness trade-offs of **rqlite**, a distributed database combining SQLite and the Raft consensus algorithm. Using a custom YCSB client and consistency fuzzer, we benchmarked rqlite across multiple replication factors (1, 3, 5) and multiple read consistency models (none, weak, linearizable, strong). Our analysis tries to quantify the throughput costs of strict distributed consensus compared to the scalability of relaxed consistency guarantees for reads.

1 Introduction

Distributed databases constantly negotiate the trade-offs between strict consistency, high availability, and performance. We examine rqlite [6], a distributed relational database that couples the ubiquitous SQLite storage engine with the Raft consensus protocol [5]. Because SQLite is inherently single-node and deterministic, rqlite effectively isolates the network and consensus overhead required by Raft to the leader node. This makes it a suitable environment for analyzing how weakening distributed consistency models and changing the replication factor can directly impact the system throughput and latency in a production-ready application.

To evaluate this, our approach measures database performance across varying degrees of fault tolerance and read consistency models. Specifically, we deployed containerized rqlite clusters with different replication factors (1, 3, and 5 nodes) and benchmarked them under four distinct read consistency models that rqlite provides: *None*, *Weak*, *Linearizable*, and *Strong*. Because standard benchmarking frameworks lack native support for rqlite’s HTTP-based SQL interface, we developed a custom driver for the Yahoo! Cloud Serving Benchmark [1]. A consistency fuzzer was also made to expose transient stale reads under heavy concurrent loads, to show the tradeoff of weakened consistency models.

Our results demonstrate that Raft’s strict fault tolerance introduces predictable performance ceilings. Enforcing *Strong*

or *Linearizable* consistency heavily throttles read operations and increases tail latency due to mandatory quorum checks. In contrast, the default *Weak* configuration provides a healthy amount of performance while remaining consistent under heavy workload demands. Additionally, the *None* configuration, while not providing any real consensus for read operations, is the only one that is able to maintain its read throughput as the cluster grows.

The remainder of this paper is organized as follows. Section 2 provides background on the underlying rqlite architecture and Raft consensus. Section 3 discusses our prior experience with distributed systems. Section 4 details our experimental methodology, custom tooling, and testing environment. Section 5 presents quantitative benchmarking results and discusses their implications. Finally, Section 6 summarizes our conclusions.

2 Background

2.1 rqlite Architecture

rqlite merges the lightweight SQLite engine with the Raft consensus protocol. Unlike databases that shard data, rqlite replicates the entire database to every node. Every individual node runs a standard unmodified instance of the SQLite database engine, with a Raft consensus layer on top of it. Writes must be routed to a single elected Raft Leader, which appends the SQL statement to its Raft log and replicates it to Follower nodes. The Leader only commits and executes the statement against the local SQLite engines after a quorum of nodes acknowledges the write, for safety reasons. This design makes writes inherently bottle-necked by network latency and synchronous disk I/O.

2.2 rqlite API and Request Format

Unlike traditional databases that rely on proprietary binary protocols and specialized client drivers (like PostgreSQL or MySQL), rqlite operates entirely over a standard HTTP/JSON

API. All database interactions – from client queries to cluster management – are executed via **standard HTTP requests**.

1. **Read Operations:** Data retrieval is typically handled via GET requests to the `/db/query` endpoint. The SQL SELECT statement is passed as a URL-encoded parameter (e.g., `q=SELECT * FROM table`).
2. **Write Operations:** Data modifications (`INSERT`, `UPDATE`, `DELETE`) are submitted via POST requests to the `/db/execute` endpoint. The SQL statements are passed within the HTTP request body as a JSON array, which allows for batching multiple operations into a single Raft log entry.
3. **Responses:** Regardless of the operation, the rqlite node replies with a JSON-formatted payload. For queries, this includes an array of the requested rows; for writes, it includes metadata such as the number of rows affected or the last insert ID.

2.3 Read Consistency Models

While rqlite’s write operations are bottle-necked by the single Raft leader, the presence of followers allows an opportunity for better read performance, at the expense of strong consistency. rqlite offers multiple read consistency levels:

1. **Strong:** The safest but least efficient option. It routes the read request entirely through the Raft log just like a write operation. Because it requires both network quorum and synchronous disk I/O across the cluster, it is significantly slower and is strictly recommended for testing rather than production code.
2. **Linearizable:** Guarantees the latest data without writing the read operation to the disk log. The Leader records the current commit index, verifies its leadership by pinging a quorum of Followers, and waits for its local SQLite engine to apply all committed writes before returning the result. Performance is bottle-necked by network latency rather than disk speed.
3. **Weak (Default):** The Leader queries its local memory without consulting Followers. This is fast, but carries a tiny risk of staleness if leadership has just changed. If a read request goes to a follower, the request is forwarded to the leader.
4. **None:** Clients can query any node (Leader or Follower) for an immediate local response, offering maximum read scalability but no guarantees against stale data.
5. **Auto:** Not a distinct consistency level, but rather a dynamic setting where the receiving node automatically selects the most sensible level based on its type (None for read-only nodes, Weak for voting nodes). **We will**

not be evaluating the Auto setting in this project, as it simply maps to the underlying core consistency levels we are already testing.

The desired read consistency level can be explicitly specified as a query string parameter when submitting GET requests (e.g., `?level=linearizable`).

2.4 Replication Factors

We evaluated rqlite over replication factors of 1, 3 and 5. While higher replication factors like 7, 9 would be useful to show a higher increase in read throughput for the “none” consistency level, we chose not to approach those numbers due to time and compute constraints.

2.5 Implementation Characteristics

The rqlite implementation is open-source and can be found at <https://github.com/rqlite/rqlite>. We deployed an unmodified official container for the rqlite repository for evaluation. These containers were deployed using Podman [8] on CloudLab [3] instances.

For load generation, we used the popular the Yahoo! Cloud Serving Benchmark (YCSB) tool. Since the standard YCSB lacks native rqlite support, we created an extension for our purposes. Additionally, a consistency fuzzer was used to stress-test the relaxed read consistency models to see how many inconsistencies would be spotted. The HTTP-centric API design eliminates the need for complex driver configurations, allowing us to easily implement the custom YCSB client and the fuzzer using standard web request libraries.

More details about this can be found in the *Methodology* section.

3 Previous Experience and Motivation

Previously, all three team members had a small amount of prior experience to the concepts used in rqlite, although none of us had worked with rqlite specifically.

Our most relevant direct experience came from the mini-projects, in which all three implemented a distributed key-value store using the MadKV [4] framework. All of us chose Raft as the consensus protocol for the project, which gave us hands-on familiarity with how it does leader election, log replication, and quorum acknowledgment function in practice. The experience made rqlite’s architecture immediately intuitive for a production scale replicated database. Additionally, two of our members, Chenglong and Daniel, used SQLite in the same mini-projects, as the durable backend store and our third member, Arjun has experience using SQLite as a webserver backend database.

Testing software artifacts on a robust testbed is an important work in this course. Two of our members, Chenglong

and Daniel, have prior experience on setting up the MadKV project on a single-node or multi-node CloudLab testing environment. Before that, Chenglong is already an experienced CloudLab user who is familiar with hardware resource reservation, multi-user environment provision and management, communication with site admin, hardware specifications, performance monitor, and network topology configuration. When provisioning user accounts on CloudLab testbed, Chenglong uses Ansible [7] to automate the process. Based on this, we plan to use CloudLab to conduct our experiments.

Deploying software artifacts correctly on the testbed is another important work in this course. All of our members have prior experience on container technology, such as Docker [2] and Podman. Container technology not only automates the software deployment process, but also makes it reproducible. This greatly reduces system administration error and helps the experiment to produce reproducible results. Rqlite is distributed through both source code and container images. We plan to deploy rqlite through Podman containers on CloudLab testbed to conduct our experiments.

One particularly memorable lesson from the mini-projects was the impact of storage hardware on Raft performance. After observing how dramatically slow HDDs were when handling the synchronous disk writes that Raft requires for log durability, we became acutely aware of how tightly consensus performance is coupled to I/O latency. This directly informed our experimental setup: we were careful to ensure that all CloudLab nodes in our cluster ran on SSDs, eliminating disk speed as an uncontrolled variable.

All these backgrounds made rqlite a natural and compelling choice for a final project. Having already built a simplified version of a Raft-replicated store ourselves, we were curious to see how a mature, production-ready system handles the same fundamental tradeoffs – particularly around how different read consistency levels can be used to trade correctness guarantees for performance.

4 Methodology

4.1 Server Topology

To keep research manageable, we created a topography of one client machine and three server machines on CloudLab. To eliminate any variables involving hardware, we made sure that every node had the same basic specs, including the SSD type. The benchmarking tests had different hardware specifications than the fuzz tests did, but in all test cases the hardware was homogenous across all machines. Figure 1 shows a diagrammatic representation of the configuration, taken straight from CloudLab. There is more information on this in the *Results and Discussion* section.

For all test cases, we ran a single cluster where we spread out the nodes across all three server machines, `storage0` to `storage2`, where rqlite is deployed. The client-side program



Figure 1: CloudLab topology showing the configuration of machines for our experiments. There are 3 storage server machines and 1 client machine connected over a switch

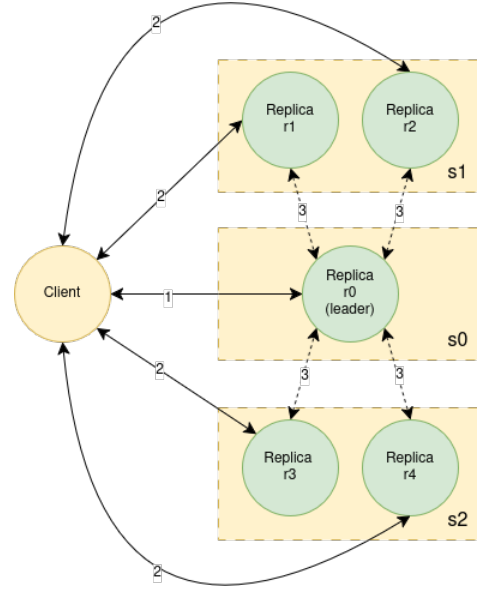


Figure 2: Rqlite replica distribution for a replication factor of 5. There are 3 server machines (`s0`, `s1`, `s2`) and 1 client machine.

issuing YCSB workload fuzz test cases is deployed on the `client` node. The rqlite node layouts on the server machines depended on the replication factor. For a factor of 1, there was only one rqlite instance on a single machine. For a factor of 3, each machine had its own rqlite node, one of which was the leader. For a factor of 5, the leader’s machine would only host the leader node and the other machines would host two follower nodes. In all cases, we made sure that the same machine would always contain the leader node.

The Cloudlab configuration for a replication factor of 5 can be seen in Figure 2. In the figure, there is only client, spawned in it’s own machine, sending requests to the rqlite system. There are 5 rqlite replicas, split across 3 machines. Replica `r0` on machine `s0` is designated as the Raft leader in this situation. Messages denoted by 1 show the requests sent by the client to the Raft leader replica. Messages denoted by 2 show the requests sent by the client to the Raft follower replicas. In consistency models, except *none*, the follower replicas

forward the requests in 2 to the leader replica. For *none* read consistency, the requests are serviced by the local replica itself. All write requests coming through 2 are forwarded to the leader replica as only it can service those requests. Messages denoted by 3 denote a) Raft messages (like `AppendEntries`, `RequestVote` etc); b) read/write requests from 2 that need to be forwarded to the leader replica.

4.2 Test Cases

We decided on multiple variables for the main set of test cases. Each row below indicates every value we used for a given variable. In both YCSB benchmarking and fuzz testing, we ran a test case for every single combination of every variable.

Parameter	Values
Replication Factor	1, 3, 5
# of Client Threads	1, 2, 4, 8, 16, 32, 64, 128, 256
Consistency Level	Strong, Linearizable, Weak, None
Workload (YCSB only)	A, B, C, D, E, F
# of Conflicting Keys (Fuzz only)	20, 60, 100, 140

Table 1: Parameters used for the experiment. Different combinations of the above are taken to evaluate performance upon.

YCSB testing made each client provide 5000 commands to the cluster. It should be noted that these numbers were accumulative and per client. This means that a YCSB benchmark running a test case with 4 threads would send 20,000 operations to the cluster. Due to having to use weaker hardware later on in the project’s runtime, we reduced the number to 2000 per client for fuzz tests.

4.3 Benchmark and Fuzzer

To interact with the `rqlite` server instances, we created a YCSB driver extension so we could run performance benchmarks under different consistency configurations. For measuring correctness and consistency, we implemented a fuzzer in Java that also uses our YCSB driver to communicate with the `rqlite` servers. We based the fuzzer closely on what already existed in the MadKV mini-projects’ repository template. Though that fuzzer was created in Rust, we used its design and concepts to create a similar version in Java that could use our driver. However, in order to more accurately determine linearizability, we tracked each operation by not only its timestamp, but by the Raft index value that `rqlite` provides with each returned operation. Depending on the consistency configuration, read-only operations may not have a Raft index number, but writes are guaranteed to have one due to always being logged.

Both the fuzzer and benchmark tests have the ability to be multithreaded, though each of these threads presents itself

to the cluster as individual, single-threaded clients. For each cluster that a client connects to, it will send commands in a round-robin fashion, starting from the first replica listed in its execution arguments. While this would be normally counterintuitive for the stronger consistency configurations that relay messages to the leader anyway, this allows for a more uniform distribution of work when testing the much weaker configurations, eliminating potential randomness otherwise present.

4.4 Experiment Automation

As shown in the previous section, for YCSB benchmark alone, there are $3 \times 9 \times 4 \times 6 = 648$ experiments to run. For fuzz tests, there are $3 \times 9 \times 4 \times 4 = 432$ experiments to run. It is unacceptable that we run each experiment involving multiple nodes manually. It is mandatory that both YCSB benchmark and fuzz tests are fully automated. Given our background, we choose Ansible to bridge this gap. Ansible is an agent-less orchestration tool responsible to automatically provision `rqlite` instances on server machines (`storage0` to `storage2`) before each experiment, and tear down `rqlite` instances on server machines after each experiment to ensure a clean slate for each run. Since Ansible is an agentless orchestration tool, it only needs provisioning on the client machine. Server machines only need to have SSH server and Python 3. We installed Ansible on the client machine and made sure SSH server and Python 3 are installed on all server machines. We also installed SSH public key of client machine on all server machines to ensure Ansible on the client machine is able to run commands on the server machines through SSH. On the server machines, we created a bash script to bring up and tear down `rqlite` nodes. Ansible issues commands to run the script on the server machines to realize provision and teardown.

Ansible needs two files to work. One is the inventory file. The other is the playbook file. The inventory file tells Ansible the location of server machines through host name, domain name or IP address. You can also tag server machines to put them into different groups. In the inventory file, we tagged `storage0` as the machine to run `rqlite` leader nodes while `storage1` and `storage2` are tagged to run follower nodes. The playbook file specifies the tasks to run on different groups of machines. We have two playbooks: One for provisioning and the other is for teardown. The provisioning playbook involves two groups: One is the leader group and the other is the follower group. This distinction is necessary because our bash script for creating and destroying nodes requires different command line arguments for the leader and followers. However, the tear down playbook does not need this distinction because the same command is used to tear down all `rqlite` nodes.

5 Results and Discussion

With our cases, we compare the performance and consistency of each major configuration that rqlite has to offer. Due to the different priorities of YCSB benchmarking and fuzz testing, we decided to keep these as two separate categories for measurement. YCSB benchmarking was done in one isolated session, and fuzz testing was done in another.

5.1 YCSB - Performance

We ran our performance benchmarks using four r6615 units from CloudLab, each of which contains a 32-core 3.25GHz CPU, an NVMe PCIe 4 SSD, and a 100Gb PCIe 4 NIC. The results that we found were, for the most part, what we expected, though not without some small surprises.

Since different workloads provided different types of operations (such as reading and writing individual rows or scanning entire subsets of the database), we decided that the best way to measure these workloads was through their average Overall Throughput as returned by YCSB. This throughput is measured in the number of operations performed per second, whether they were write operations guaranteed to be logged or read operations that could be treated differently depending on the test case. As shown in Figure 3, our initial expectations for the order of consistency were proven correct. Strong, following the Raft protocol in the strictest sense, performed the slowest of the four major configurations. Linearizable was the second slowest, followed by Weak being the second fastest. It was no surprise to us that None, the only configuration that makes no effort to coordinate read operations between nodes, was the fastest of all the configurations.

Each consistency level had a best workload that outperformed the others, but what surprised us was that said best workload was not the same for every consistency level. Strong’s best workload was A, an even distribution of individual reads and writes. Linearizable’s best workload was B, which consisted of mostly individual reads with some writes. Weak and None shared a best workload of C, exclusively consisting of individual reads. Weak and None’s choice in workload was expected, since their read operations had the least overhead and it would make sense that they would be fastest in a read-only workload. Rather predictably, None was also the only configuration that increased its read throughput as the number of nodes increased. Linearizable’s choice also made sense to us, because while it has more read overhead than Weak and None, it still only has to check with the cluster that the latest write has been committed before checking its local state.

The major surprise to us was that Strong had a “preferred” workload at all. In Strong, every operation, read or write, is logged and run through the whole Raft protocol. We thought that the strictness of the setup would make every workload be treated the same way (except for workloads E and F which

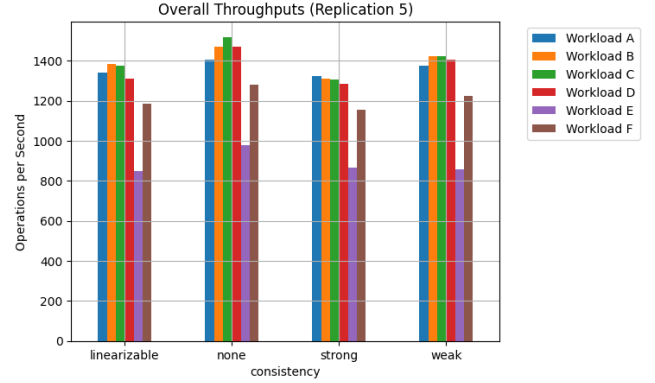


Figure 3: Overall throughput of rqlite under the replication factor of 5. 4 different read consistency levels (none, weak, linearizable, and strong) and 6 YCSB workloads (A-E) are covered.

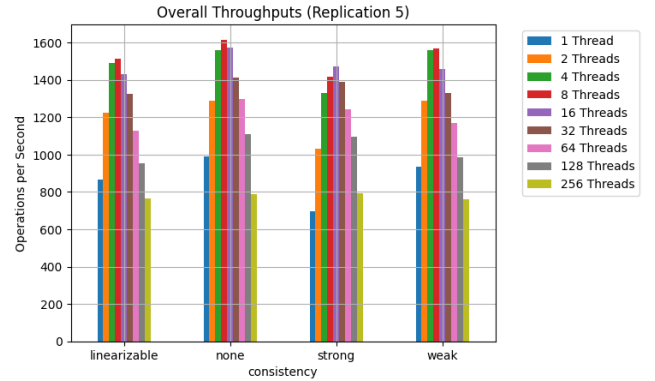


Figure 4: Overall throughput of rqlite under the replication factor of 5. 4 different read consistency levels (none, weak, linearizable, and strong) and 9 concurrency levels are covered.

use complex operations). Given how small the margins are, it is possible that this may be less a definitive best workload and more a matter of small variance that would likely change under more data.

Overall, Figure 4 shows that the cluster would eventually grow saturated as the number of threads increased. In a replication factor of five, most consistency configurations were able to provide the most throughput when they were handling around eight clients. Strong consistency was the exception, optimally handling sixteen clients. Once the number of clients increased, the cluster was congested and performance started to degrade even before the machines themselves could have been saturated. As each individual machine had 64 physical threads (meaning we had 64 client threads and 192 server threads at our disposal), we know this was not a hardware issue but a result of rqlite’s consensus protocol. This bottleneck could be solved by having multiple clusters with disjoint key partitions, but the explicit purpose of our project was to strain

one cluster in the hopes of causing consistency issues.

5.2 Fuzz - Correctness

We had originally planned to run fuzz testing on the same exact hardware that we used to gather our YCSB benchmarks. Unfortunately, we discovered bugs in our history tracker that caused many false positives to appear, ruining our correctness data. By the time we fixed the issues, we were unable to reserve them again. Instead, we went with a set of comparatively weaker `c220g2` units. Each of these units had a 10-core 2.60 GHz CPU, a 10K RPM HDD, a SATA SSD, and 10GB PCIe 3 NIC.

Our fuzz test results, while accurate, were not as diverse as we were hoping them to be. Throughout our testing, no matter the replication factor applied, the only test cases that ever saw any consistency issues were those specifically set to the “None” read consistency configuration. The count of inconsistent operations per replication factor can be seen in Table 2, but these are all attributed to None. All other configurations (Strong, Linearizable, and Weak) all had no measurable inconsistencies. We had hoped that by stressing the cluster with enough threads and conflicting write requests, we could cause some slight faltering in the “Linearizable” or “Weak” configurations. However, these configurations were able to hold up in spite of our efforts.

Regarding the None-category test cases, we found that, as one would likely expect, the number of consistency violations drastically increases with the number of existing nodes. This was what we expected from the beginning, as the setting means that read operations are not coordinated in any way, with only write operations being shared with any sort of care.

Replication Factor	Total Inconsistent Operations
1	0
3	751,157
5	1,222,743

Table 2: Impact of Replication Factor on operation consistency under no enforced read consistency.

We tried to tamper with the system in a few ways in hopes of seeing a less skewed distribution of failures, but nothing we did was able to cause failures to Linearizable or Weak test cases.

First, we attempted to add artificial latency to all node-to-node network communication, the idea being that enough latency could force re-elections and potentially cause inconsistencies in those two settings. However, we were unable to do these latency tests at scale for two reasons. The first was that we couldn’t find the right amount of latency that would cause the right conditions to trigger inconsistencies. Too little latency and it would have no meaningful effect. Too much latency and the nodes would never agree on a leader

at all, preventing any tests from running. The second reason was that, even if we had found the ideal conditions to cause these breaks, our test cases in general could not scale with the latency. There are millions of operations per key-set. Each key-set is run multiple times at different consistency levels, and these consistency-level tests are repeated at three different replication factors. Even a small artificial delay would cascade into the runs taking exponentially longer. We simply did not have the time for that. We tried one specific test case with exactly one set of variables as covered in *Limitations*.

The second major method we tried was doing a full run with the nodes’ storage being placed in a RAM-based filesystem. The idea was that the increase in write speed from this might be something that `rqlite` might not have accounted for, potentially causing instability in the cluster. In practice, however, we had no faster testing and as a result had no meaningful effect. None of the three configurations with actual consistency ever received errors. In hindsight, it’s unlikely that this would have mattered even with the more powerful setup, given that the RAM-based filesystem uses the same underlying API as any filesystem hosted on an HDD or SSD.

5.3 Limitations

Although both YCSB benchmarking and fuzz testing are conducted on a multi-node infrastructure, we note that the round-trip time (RTT) between any two nodes (client-server, or server-server) are near-zero because all of them are located in the same LAN. For the `r6615` cluster used for YCSB benchmarking, the RTT revealed by ping is around 0.07 ms. For the `c220g2` cluster used for fuzz testing, RTT revealed by ping is around 0.1 ms. For a real-world `rqlite` deployment, such latency is too ideal. Considering this factor may cover up `rqlite`’s behavior under some corner cases, we decided to impose some artificial latency plus packet drop with Linux traffic control (`tc`) utility. Given time constraints, we are not able to run a full fuzz test, given that a full fuzz test with ideal network conditions already needs around 4 hours to run. Scaling up latency to 4 orders of magnitude and amplifying it with millions of operations will make us run out of time. Therefore, we just run a single fuzz test case with a replication factor of 3, weak consistency, and 32 threads. In our first attempt, we imposed a 300 ms latency with 20 ms jitter plus a 2% packet drop rate on all **outbound** traffic for all server nodes (`storage0` to `storage2`). However, such extreme network conditions made `rqlite` nodes unable to reach a consensus to find the leader to get the fuzz test started. In our second attempt, we reduced such latency to 150 ms. Although the fuzz test was able to run, it passed the consistency check. We concluded that such network conditions are not able to stably reveal the `rqlite`’s corner case and results in inconsistent reads. Moreover, this fuzz test is not a full run. Given these considerations, we decided to just include the fuzz test result collected under ideal network conditions in our report. For

future experiments, it may be necessary to include both network latency and network partitions to reveal the inconsistent read of rqlite under weak consistency level.

6 Conclusion

Overall, our throughput benchmark testing was effective in measuring how each consistency level differs from one another. Each configuration had different workloads that it got the most throughput from. Weighed under said workloads, there was also a clear pattern in how quickly each configuration could work depending on its consistency level. Strong was very clearly the slowest, followed by Linearizable and Weak, with None being obviously the fastest of the four. With that being said, our fuzz testing left much to be desired. While we created a valid checker for operation linearizability, we were unable to engineer circumstances that would have caused inconsistencies in the Linearizable or Weak configurations such that they would contain different results than Strong. Given more time, we likely would have been able to expand on our test cases and experiment with more rqlite configurations in order to create more instability and potentially more interesting results.

References

- [1] Brian Cooper. Yahoo! cloud serving benchmark, 2019. Accessed: 2026-05-07.
- [2] Docker, Inc. Docker, 2026. Accessed: 2026-05-07.
- [3] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The design and operation of CloudLab. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 1–14, July 2019. Accessed: 2026-05-07.
- [4] Guanzhou Hu and Junxuan Liao. Madkv, 2026. Accessed: 2026-05-07.
- [5] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, USENIX ATC’14, page 305–320, USA, 2014. USENIX Association. Accessed: 2026-05-07.
- [6] Philip O’Toole. rqlite: The lightweight, distributed relational database with raft consensus, 2026. Accessed: 2026-05-07.
- [7] Red Hat, Inc. Ansible community documentation, 2026. Accessed: 2026-05-07.
- [8] The Podman Team. Podman, 2026. Accessed: 2026-05-07.